

Machine Learning

Lecture 2: Linear Models

Iurii Efimov



Outline

1. Linear models overview
2. Linear Regression under the hood
3. Gauss-Markov theorem
4. Regularization in Linear regression
5. Model validation and evaluation

Previous lecture recap



- Dataset, observation, feature, design matrix, target
- i.i.d. property
- Model, prediction, loss/quality function
- Parameter, Hyperparameter



Supervised learning problem statement

Training set $\mathcal{L} = \{\mathbf{x}_i, y_i\}_{i=1}^n$, with $\mathbf{n}$ objects each having $\mathbf{p}$ features, where

- $(\mathbf{x}_i \in \mathbb{R}^p, y_i \in \mathbb{R})$ for regression
- $(\mathbf{x}_i \in \mathbb{R}^p, y_i \in \{-1, +1\})$ for binary classification

Model $\hat{y} = f(\mathbf{x})$ predicts some value $\hat{y}$ for every object

Loss function $Q(\mathbf{x}, y, \hat{y}, f)$ that should be minimized



Unsupervised learning problem statement

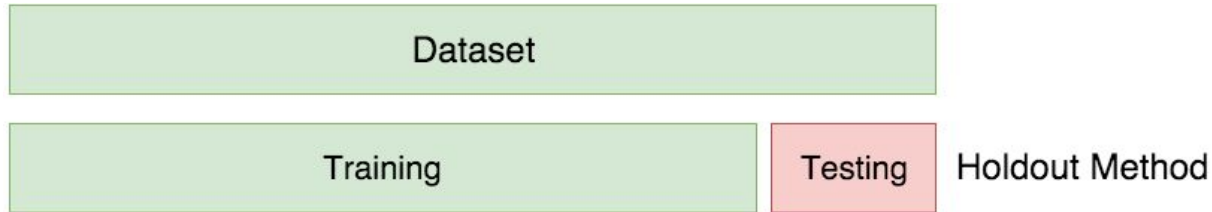
Training set $\mathcal{L} = \{\mathbf{x}_i\}_{i=1}^n$, with $\mathbf{n}$ objects each having $\mathbf{p}$ features,

where $\mathbf{x}_i \in \mathbb{R}^p$

Model $\hat{y} = f(\mathbf{x})$ predicts some value $\hat{y}$ for every object

Loss function $Q(\mathbf{x}, \hat{y}, f)$ that should be minimized

Evaluating the quality (simple)



Quality != loss function

Image credit: Joseph Nelson [@josephofiowa](#)

Regression model

girafe
ai

01

Regression model



Can be written in form

$$Y = f(X) + \varepsilon$$

or equivalently

8

$$\mathbb{E}(Y|X) = f(X)$$

here we split all the signal measured (Y) into parts containing:

1. Expectation with zero Variance (which we want to predict)
2. Variance with zero Expectation (aka noise)



Regression model assumptions



1. The sample is representative of the population at large
2. The independent variables are measured with no error
3. Deviations from the model have an expected value of zero, conditional on covariates: $E(e_i|X_i)=0$
4. The variance of the residuals e_i is constant across observations (homoscedasticity) 9
5. The residuals e_i are uncorrelated with one another. Mathematically, the variance–covariance matrix of the errors is diagonal.

See https://en.wikipedia.org/wiki/Regression_analysis#Underlying_assumptions and https://en.wikipedia.org/wiki/Additive_white_Gaussian_noise



Linear Models

girafe
ai

01

Linear models



In applied statistics we use linear functions to predict different values

$$Y = X_1 + X_2 + X_3$$

Dependent Variable

Independent Variable

Outcome Variable

Predictor Variable

Response Variable

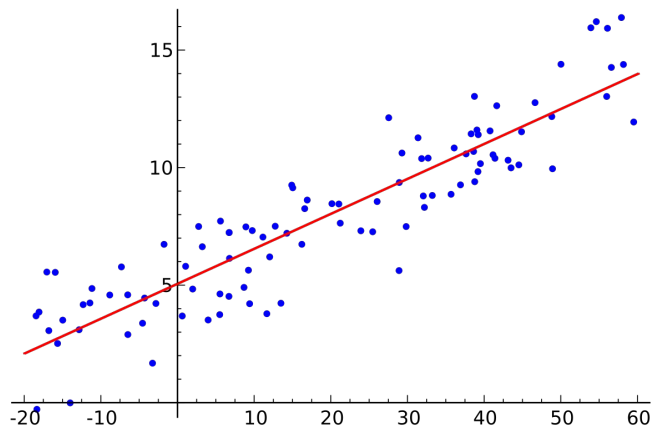
Explanatory Variable



Linear models



- Regression models



Estimated
(or predicted)
Y value for
observation i

Estimate of
the regression
intercept

Estimate of the
regression slope

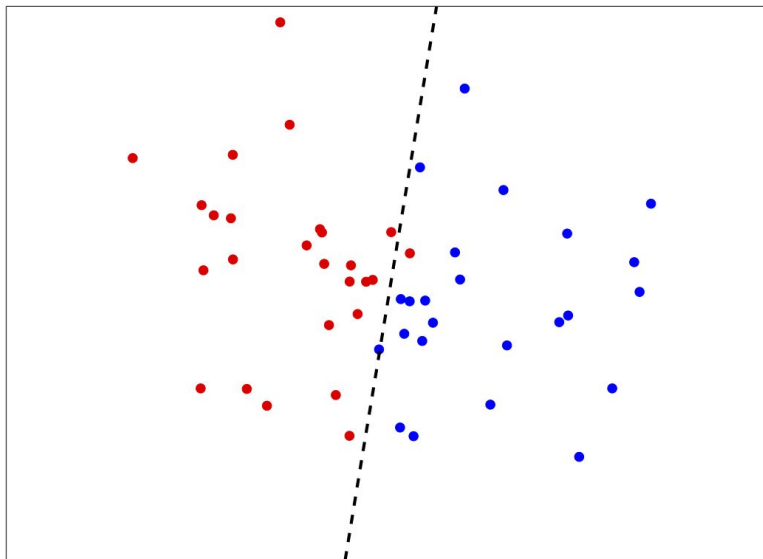
Value of X for
observation i

$$\hat{Y}_i = b_0 + b_1 X_i$$

Linear models



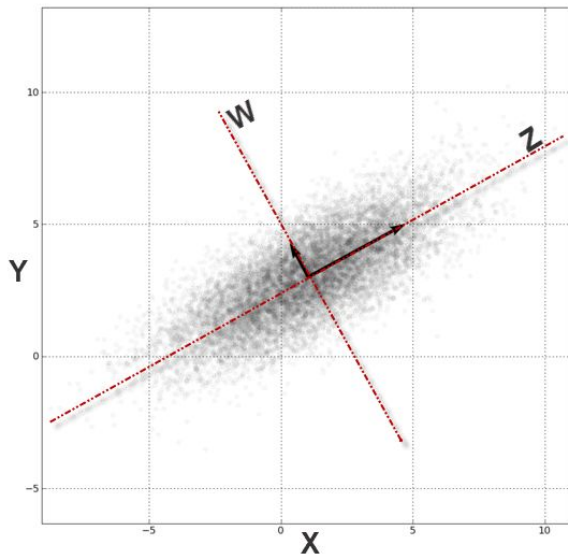
- Regression models
- Classification models



Linear models



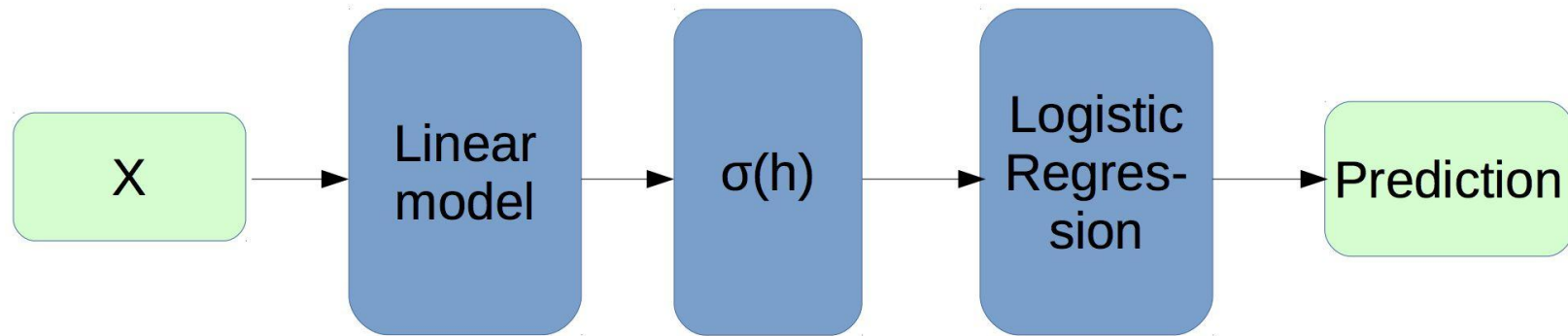
- Regression models
- Classification models
- Unsupervised models (e.g. PCA analysis):





Linear models

- Regression models
- Classification models
- Unsupervised models (e.g. PCA)
- Building block of other models (ensembles, NNs, etc.)



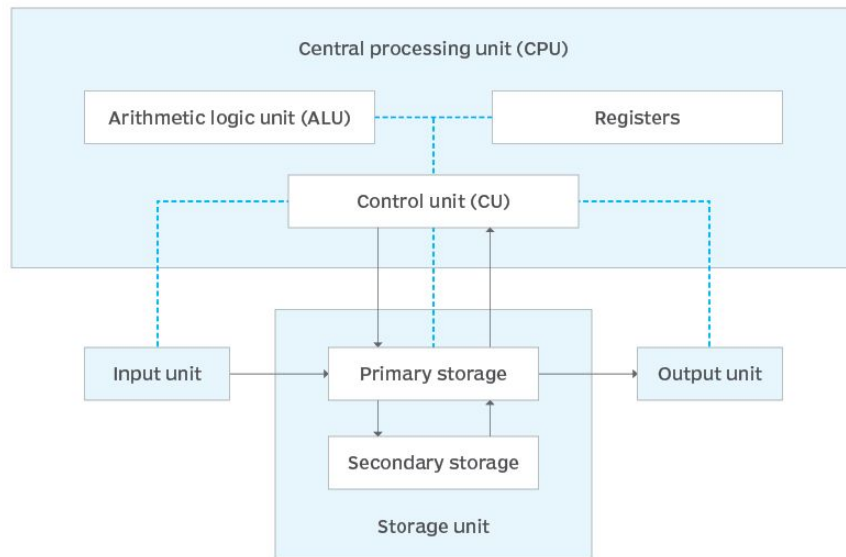
A simple neural network

The only efficiently calculated

For computers the only one model we can calculate natively is linear model.

That's why it is so important to study it in depth!

Conceptual overview of a computer system



Linear Regression

girafe
ai

02

Linear Regression model



When estimator is linear

$$f_w(x) = w_0 + \sum_{i=1}^p w_i x_i \equiv x^T w$$

Note: x and w are supposed to include bias term (conventional notation in math)

$$w = (w_0, w_1, \dots, w_n)^T$$

$$x = (1, x_1, \dots, x_n)^T$$

Linear Regression problem



Observed samples

$$(x^i, y^i), i = 1, \dots, n$$

$$x^i \in R^p, y^i \in R$$

Matrix form of dataset

$$X = [x^1, \dots, x^n]^T, X \in R^{n \times p}$$

$$Y = [y^1, \dots, y^n]^T, Y \in R^n$$

Linear Regression

$$f_w(X) = Xw = \hat{Y} \approx Y$$

How to find parameters vector?



Apply MLE estimation!

[Great visual explanation](#)

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

Linear Regression problem



How to choose weights?

With MLE!

Empirical risk = $\sum_{\text{by objects}}$ Loss on object $\rightarrow \min_{\text{model params}}$

$$Q(X) = \sum_{i=1}^n L(y^i, f_w(x^i)) \rightarrow \min_w$$

Loss functions

MSE: $L(y_t, y_p) = (y_t - y_p)^2$

MAE: $L(y_t, y_p) = |y_t - y_p|$

MSE minimization equivalents
Maximum Likelihood Estimation in
certain conditions (e.g. Gaussian noise)

Linear Regression analytical solution



For MSE closed form solution exists

$$Q_{\text{MSE}}(X) = \sum_{i=1}^n (y^i - f_w(x^i))^2 = \|Y - Xw\|^2 = (Y - Xw)^T (Y - Xw) \rightarrow \min_w$$

$$\begin{aligned}\nabla_w Q(X) &= \nabla_w (Y^T Y - (Xw)^T Y - Y^T Xw + (Xw)^T Xw) = \\ &= 0 - Y^T X - Y^T X + 2X^T Xw^T = 0\end{aligned}$$

$$w^* = (X^T X)^{-1} X^T Y$$

Gauss-Markov theorem

girafe
ai

03



Gauss-Markov theorem

$$Y = f(X) + \varepsilon$$

$$f_w(x) = w_0 + \sum_{i=1}^p w_i x_i$$

$$\mathbb{E}(\varepsilon_i) = 0 \quad \forall i$$

$$\text{Var}(\varepsilon_i) = \sigma^2 < \inf \quad \forall i$$

$$\text{Cov}(\varepsilon_i, \varepsilon_j) = 0 \quad \forall i \neq j$$

Minimizing MSE loss gives

Best Linear Unbiased Estimation (BLUE)

Estimator with minimal Variance from all unbiased linear estimators

$$w^* = (X^T X)^{-1} X^T Y$$

$$\mathbb{E}(w^*) = w_{\text{true}}$$

$$\text{Var}(w^*) = \min$$

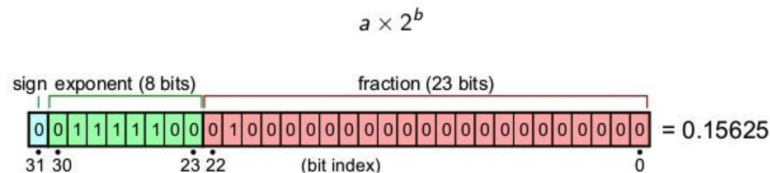


Problems with matrix solution

- Computation complexity
 - $\sim n^3$ for $n \times n$ matrix
- Unstable inversion result
 - fixed point arithmetics
 - multicollinear features

$$w^* = (X^T X)^{-1} X^T Y$$

So no one uses such way to find solution.



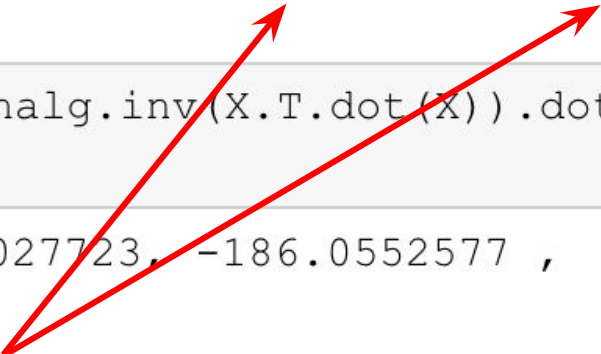


Unstable solution

In case of multicollinear features the matrix $X^T X$ is almost singular .

It leads to unstable solution:

```
w_true  
array([ 2.68647887, -0.52184084, -1.12776533])  
  
w_star = np.linalg.inv(X.T.dot(X)).dot(X.T).dot(Y)  
w_star  
array([ 2.68027723, -186.0552577 , 184.41701118])
```



corresponding features are almost collinear



Unstable solution

In case of multicollinear features the matrix $X^T X$ is almost singular .

It leads to unstable solution:

```
w_true  
array([ 2.68647887, -0.52184084, -1.12776533])  
  
w_star = np.linalg.inv(X.T.dot(X)).dot(X.T).dot(Y)  
w_star  
array([ 2.68027723, -186.0552577 , 184.41701118])
```

the coefficients are huge and sum up to almost 0

Regularizations

girafe
ai

04



L2 regularization

To make the matrix nonsingular, we can add a diagonal matrix:

$$w = (X^T X + \lambda^2 I)^{-1} X^T Y$$

It is also called Tikhonov regularization or [Ridge regression](#).

Regularization constraints model weights and decrease them.

Resulting solution is **biased** (doesn't converge to true value of w^*).

Actually, it's a solution for the following loss function:

$$L_2 = ||Y - Xw||_2^2 + \lambda^2 ||w||_2^2$$

exercise: derive it by yourself



L2 regularization properties

$$w = (X^T X + \lambda^2 I)^{-1} X^T Y$$

$$L_2 = ||Y - Xw||_2^2 + \lambda^2 ||w||_2^2$$

Properties:

- prevents overfitting
 - e.g. outliers in data will affect solution less
- penalizes weights, so they decrease
 - higher weights receive more decrease

L1 regularization



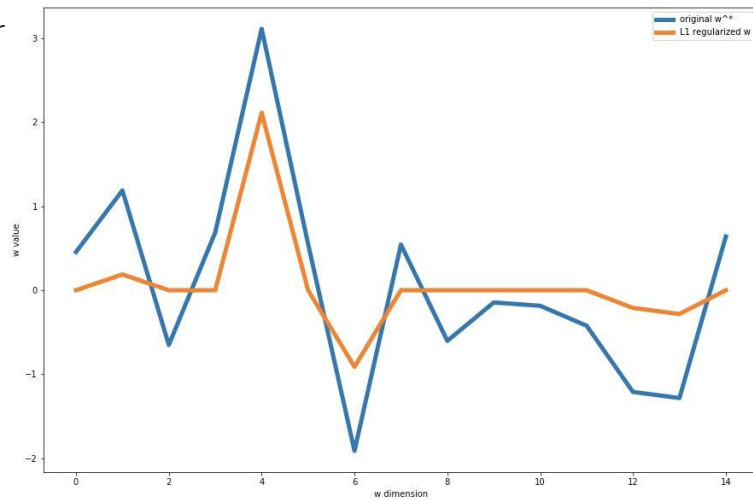
What if we add L1 norm of w to our loss? This technique is called [LASSO](#): least absolute shrinkage and selection operator

$$L_1 = ||Y - Xw||_2^2 + \lambda^2 ||w||_1$$

For this case there is no such an elegant solution as for L2 regularization, however solution exists for orthonormal design (see Spokoiny's [book](#) p.173 or [this post](#))

$$\hat{\theta}_j = \begin{cases} (\tilde{\theta}_j - \lambda)_+ & \tilde{\theta}_j \geq 0, \\ -(|\tilde{\theta}_j| - \lambda)_+ & \tilde{\theta}_j < 0 \end{cases}$$

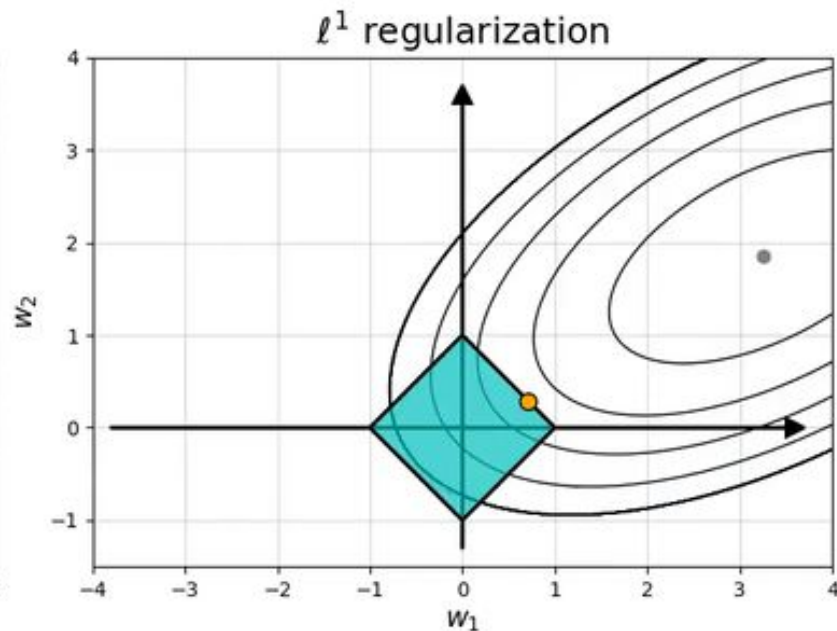
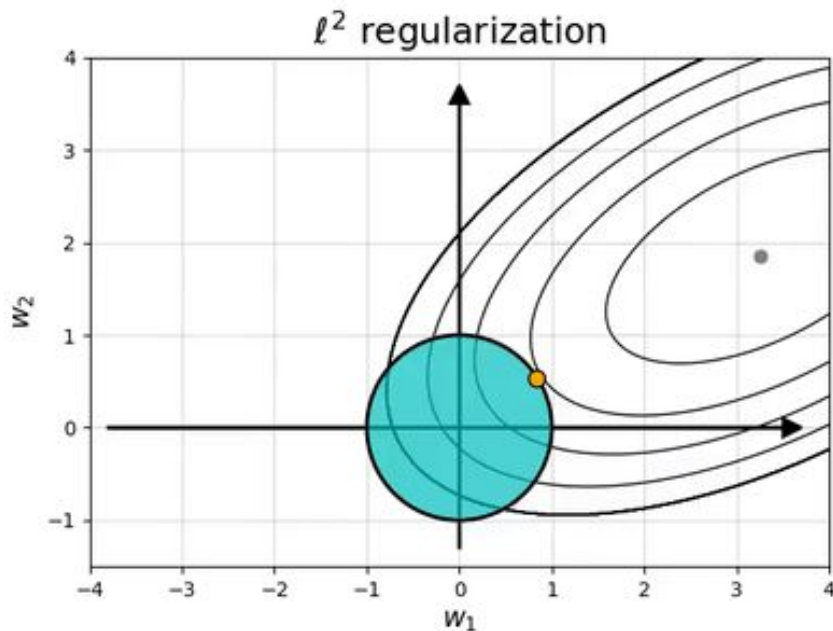
Thus this type of regularization performs implicit feature selection



L1 vs L2 regularization



ℓ^1 induces sparse solutions for least squares



New optimum point is a balance between original problem and regularization.
New solution is biased!

ElasticNet regularization



Applying both types of regularization also works

$$L_{EN} = ||Y - Xw||_2^2 + \lambda_1^2 ||w||_1 + \lambda_2^2 ||w||_2^2$$

It has properties of both types of regularizations at the same time



Regularization of bias term

Let's return to full definition of linear regression:

$$f_w(x) = w_0 + \sum_{i=1}^p w_i x_i \equiv x^T w$$

What should we do with the constant term w_0 in context of regularization?
Why?

Bias term is **not penalized** by regularization.

Why this all works?

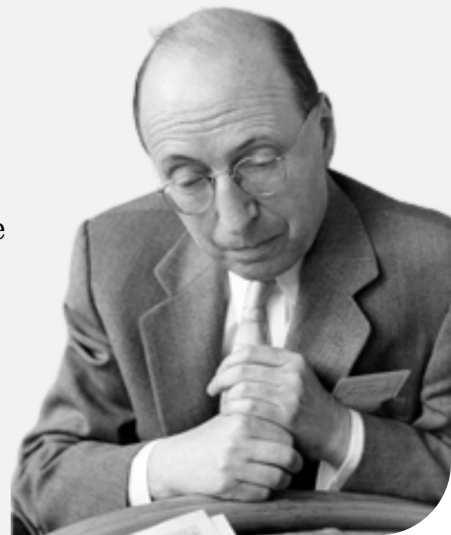


Regularization could be derived from various considerations.

Neither of these is single source of truth roots of this technique. Each of them is a valid motivation and all of them hold at the same time.

Such type of encounters are covered in classical article **“The Unreasonable Effectiveness of Mathematics in the Natural Sciences”** by Eugene Wigner written in 1960. [Text](#), [Wikipedia](#)

“The enormous usefulness of mathematics in natural sciences is something bordering on the mysterious, and there is no rational explanation for it. It is not at all natural that "laws of nature" exist, much less that man is able to discover them. The miracle of the appropriateness of the language of mathematics for the formulation of the laws of physics is a wonderful gift which we neither understand nor deserve”





Loss functions in regression

$$MSE(\mathbf{y}, \hat{\mathbf{y}}) = \frac{1}{N} \|\mathbf{y} - \hat{\mathbf{y}}\|_2^2 = \frac{1}{N} \sum_i (y_i - \hat{y}_i)^2$$

$$MAE(\mathbf{y}, \hat{\mathbf{y}}) = \frac{1}{N} \|\mathbf{y} - \hat{\mathbf{y}}\|_1 = \frac{1}{N} \sum_i |y_i - \hat{y}_i|$$



Different norms

Once more: loss functions:

$$MSE = \frac{1}{n} \|\mathbf{x}^T \mathbf{w} - \mathbf{y}\|_2^2$$

only works for Gauss-Markov theorem

$$MAE = \frac{1}{n} \|\mathbf{x}^T \mathbf{w} - \mathbf{y}\|_1$$

Regularization terms:

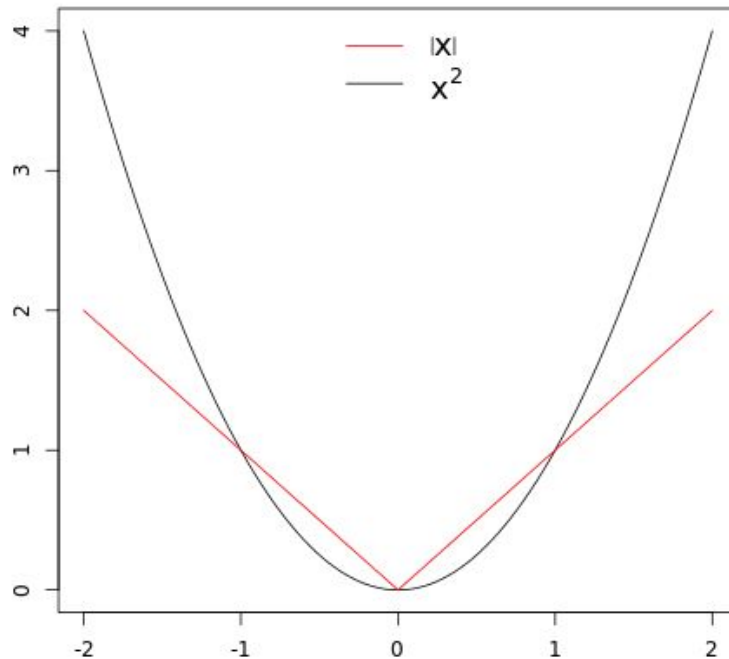
- $L_2 \quad \|\mathbf{w}\|_2^2$

- $L_1 \quad \|\mathbf{w}\|_1$

Loss function properties



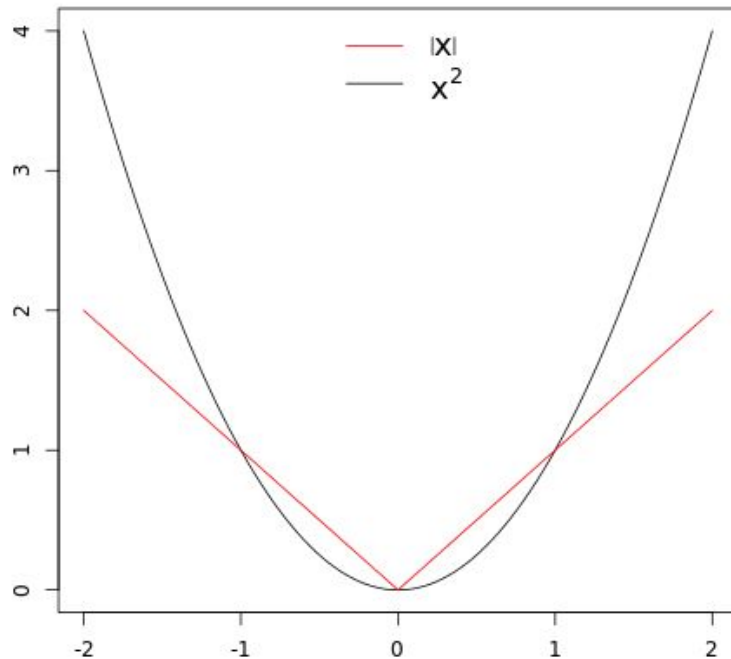
- MSE (L_2)
 - delivers BLUE according to Gauss-Markov theorem
 - differentiable
 - sensitive to noise
- MAE (L_1)
 - non-differentiable
 - not a problem
 - much more prone to noise



Regularization properties



- L2 regularization
 - constraints weights
 - delivers more stable solution
 - differentiable
- L1 regularization
 - non-differentiable
 - not a problem
 - selects features



Metrics in regression

girafe
ai

06

Metrics in regression



- MSE - Mean Square Error
- MAE - Mean Absolute Error
- RMSE - Root Mean Square Error
- MAPE - Mean Absolute Percentage Error
- SMAPE - Symmetric Mean Absolute Percentage Error
- R2 - “R squared” aka coefficient of determination
- etc... (any combination you like)

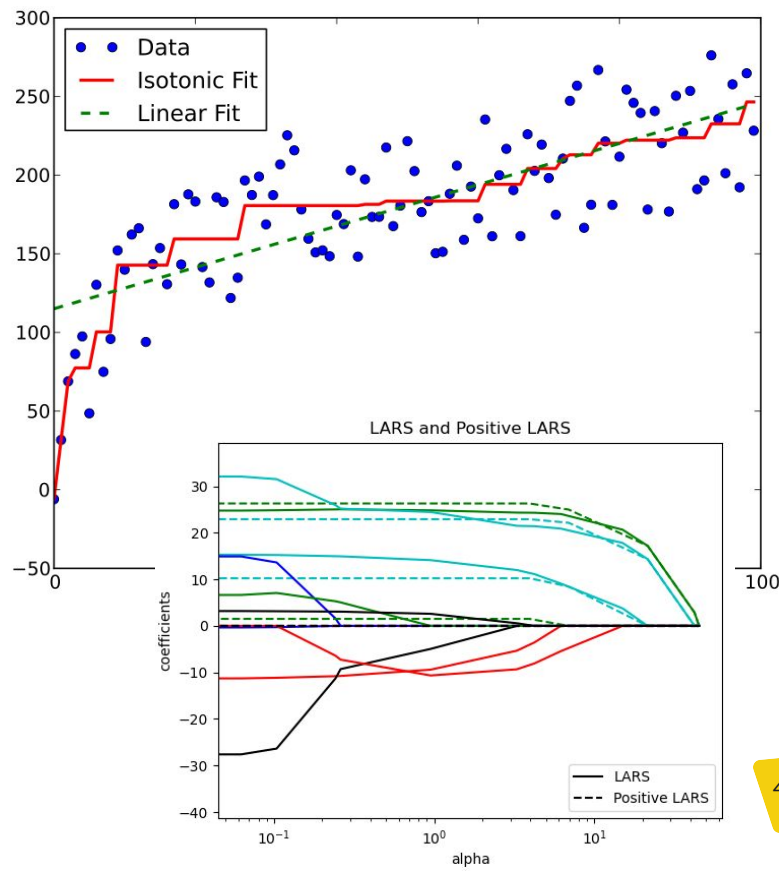
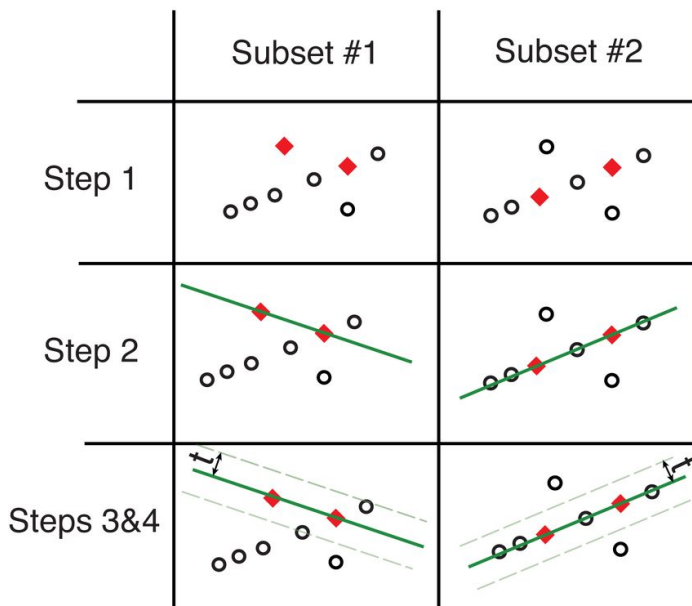
$$\text{MAPE} = \frac{1}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right|$$

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2} \quad \text{SMAPE} = \frac{1}{n} \sum_{i=1}^n \frac{2 \cdot |y_i - \hat{y}_i|}{|y_i| + |\hat{y}_i|}$$

Other types of regression models



- [Isotonic regression](#)
- [RANSAC](#)
- [Least-angle regression](#)
- [GLM](#) ([video](#))



Model validation and evaluation

girafe
ai

06

Parameters and hyperparameters



The values of parameters are derived via learning. In other words parameter values are result of optimization process.

A hyperparameter is a constant parameter whose value is set before the learning process begins.

Technically any parameter can be made hyperparameter (by fixing it) and vise versa (by estimating hyperparameter from data anyhow).

But for mainstream models there is a stable setup on what is what.



Hyperparameter examples

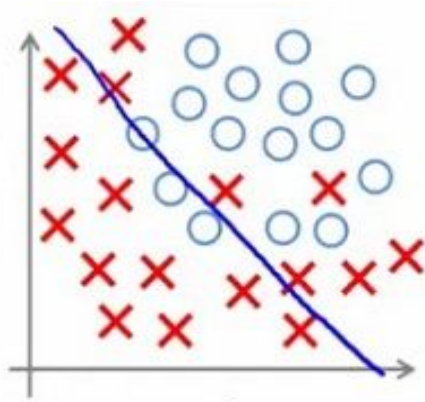
- Naive Bayes classifier - distribution of a feature
 - e.g. Gaussian, Uniform, Multinomial, etc
- Linear regression - regularization type and coefficient
 - e.g. L1, L2 and value of regularization constant
- Decision trees - max depth of a tree
- Neural networks - number of layers

Parameters vs Hyperparameters



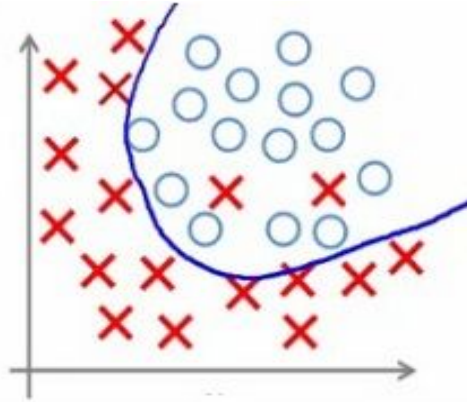
	Defined by	Depend on the training data	Order of optimization methods	Required for	Affect the capacity of the model
Parameters	during the training	yes	first (gradient)	predictions	no
Hyperparameters	before the start of training	no	zero (manual, Bayesian)	training	yes

Overfitting vs underfitting

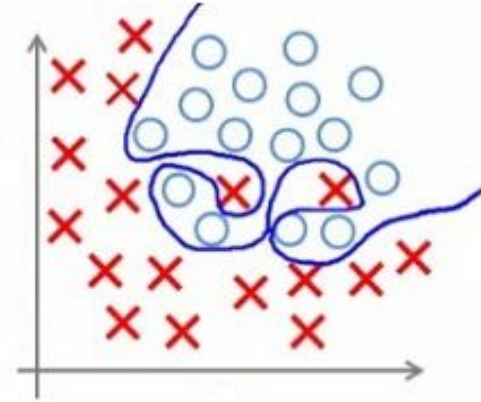


Under-fitting

(too simple to
explain the
variance)



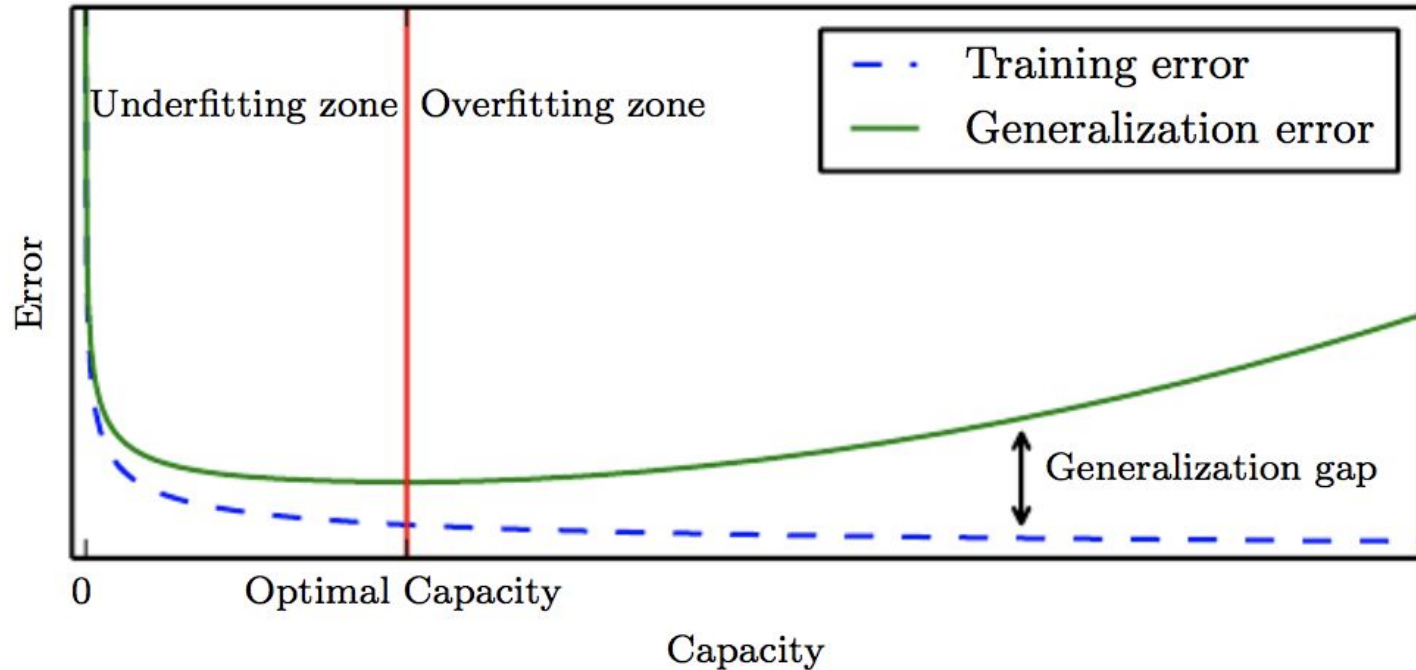
Appropriate-fitting



Over-fitting

(forcefitting -- too
good to be true)

Overfitting vs. underfitting



Evaluating the quality



Dataset

Training

Testing

Holdout Method

Is it good enough?
Could something go wrong?

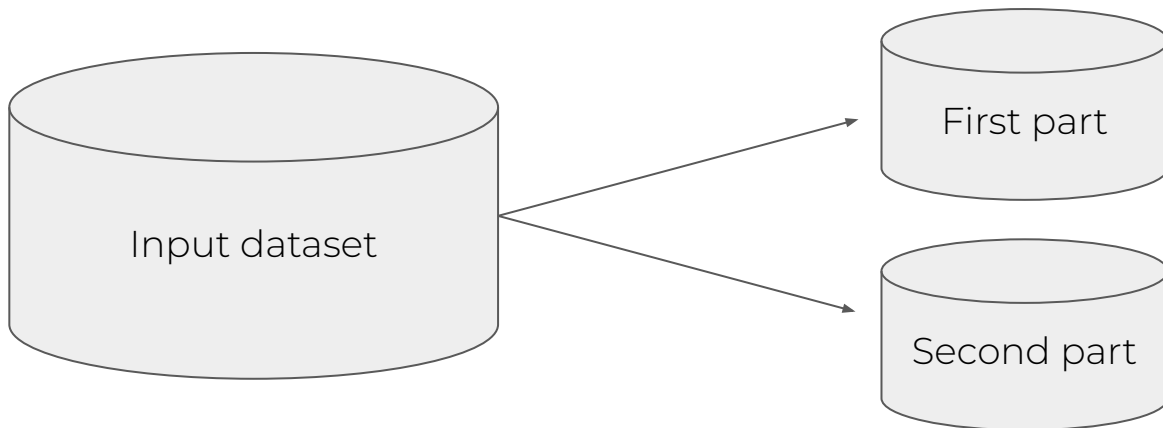


Split methods

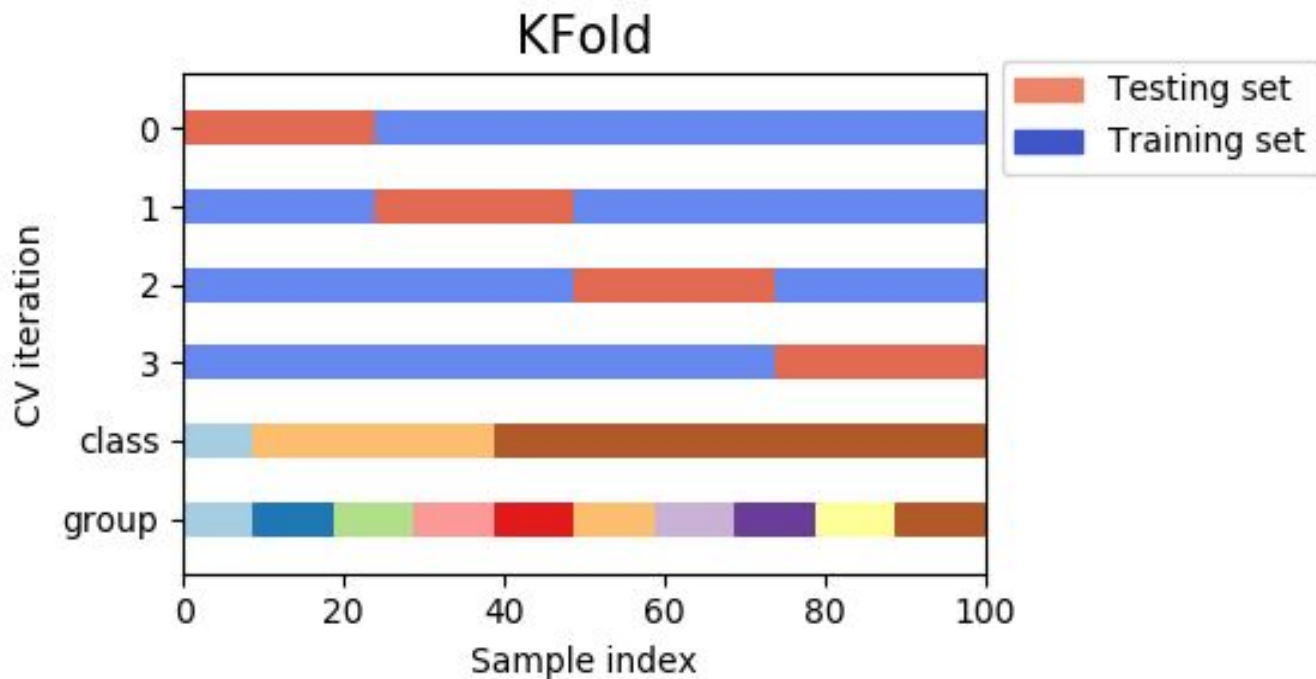
are algorithms to split one set to two sets of predefined size which is *different from stages organization* approaches described above.

Ideal requirement for a split method is **matching of data distributions** in resulting sets.

But in practice we can't check distribution, so we narrow our requirement down to **matching of selected statistics**.

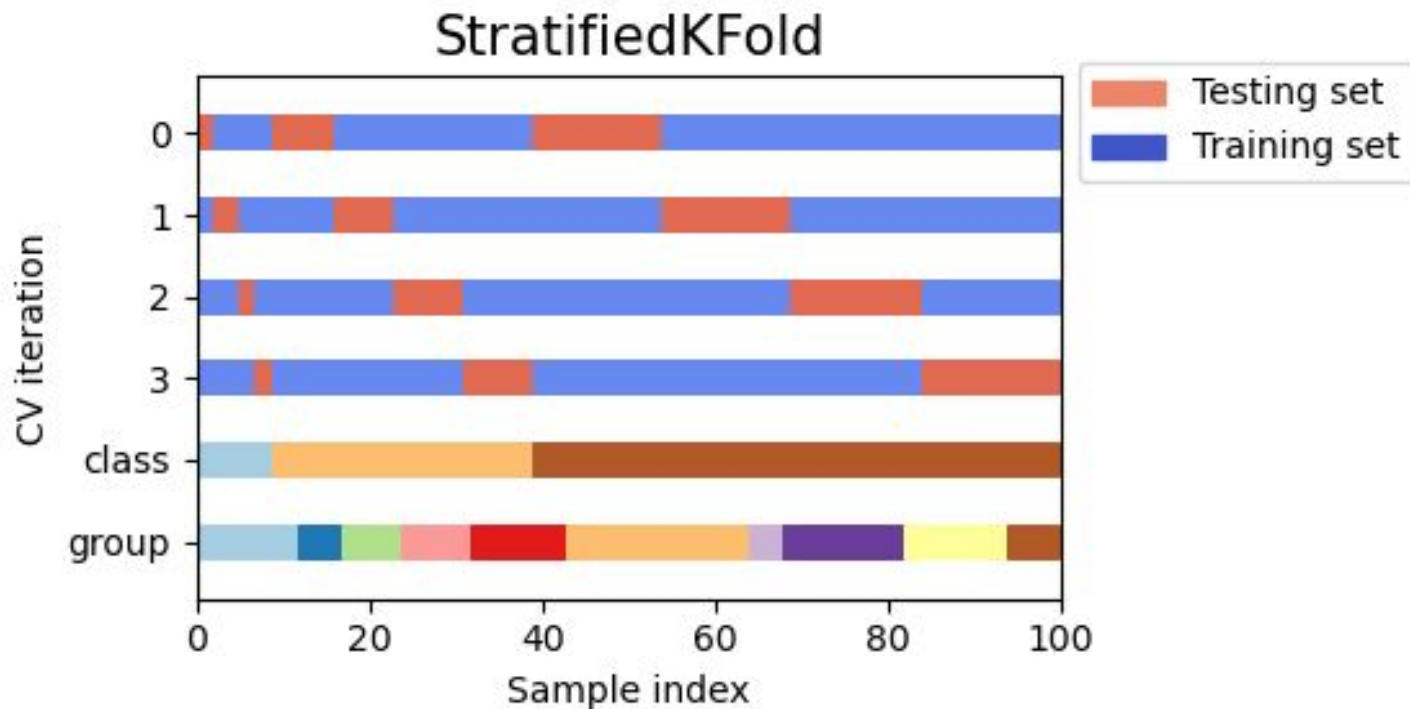


Uniform random split



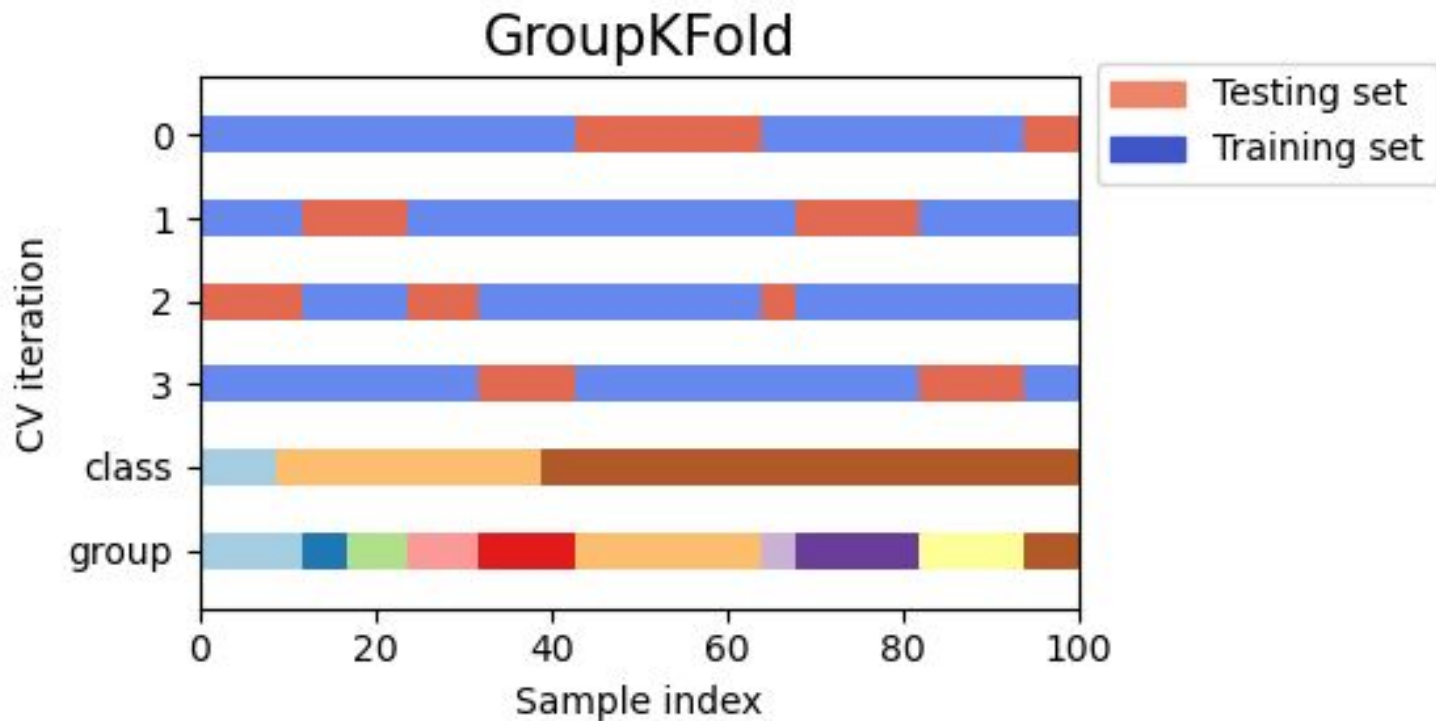
Baseline method. Word *uniform* is very important!

Stratified split



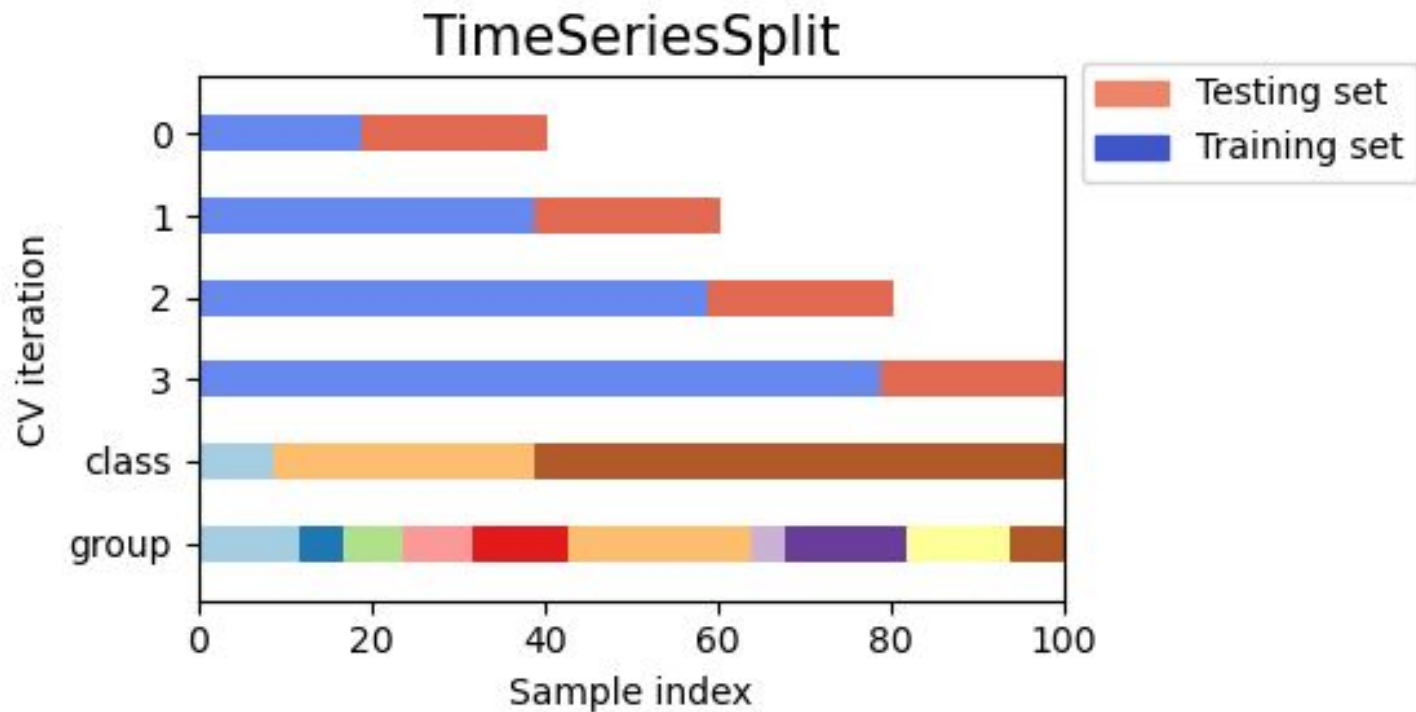
Preserve class ratio for each split. **Default for sklearn** methods

Grouped split



Set whole group either to train or validation.
Used in medicine and ranking (search, recsys)

Timeseries split

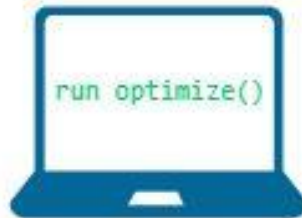


Never use train_test_split in this case!!!

Evaluation procedure



Hyperparameters



Parameters



Score

-  `n_layers = 3`
`n_neurons = 512`
`learning_rate = 0.1`
-  `n_layers = 3`
`n_neurons = 1024`
`learning_rate = 0.01`
-  `n_layers = 5`
`n_neurons = 256`
`learning_rate = 0.1`



Weights
optimization

Weights
optimization

Weights
optimization



85%

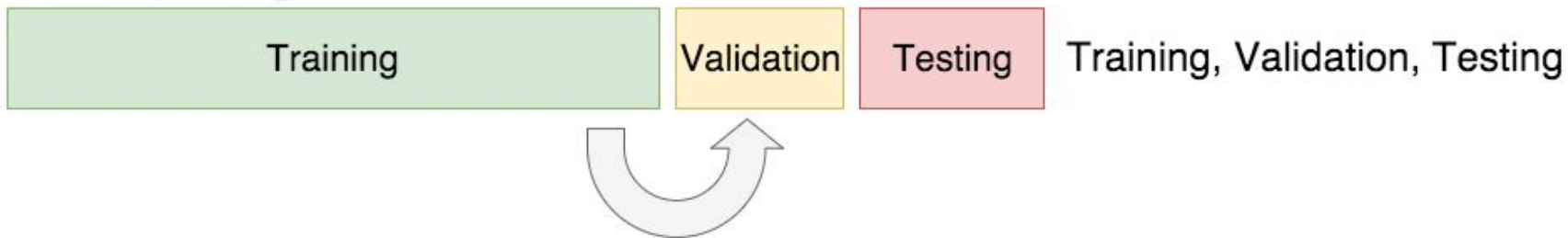
80%

92%

Dataset splits



Data Permitting:



Fixed dataset split is the most used in practice nowadays because sizes of dataset are usually big enough.

Stages of model creation



Split	training	validation	test
Used for	parameters optimization	hyperparameters selection	quality measurement
Overfitting level	high	average	low

The most common mistake

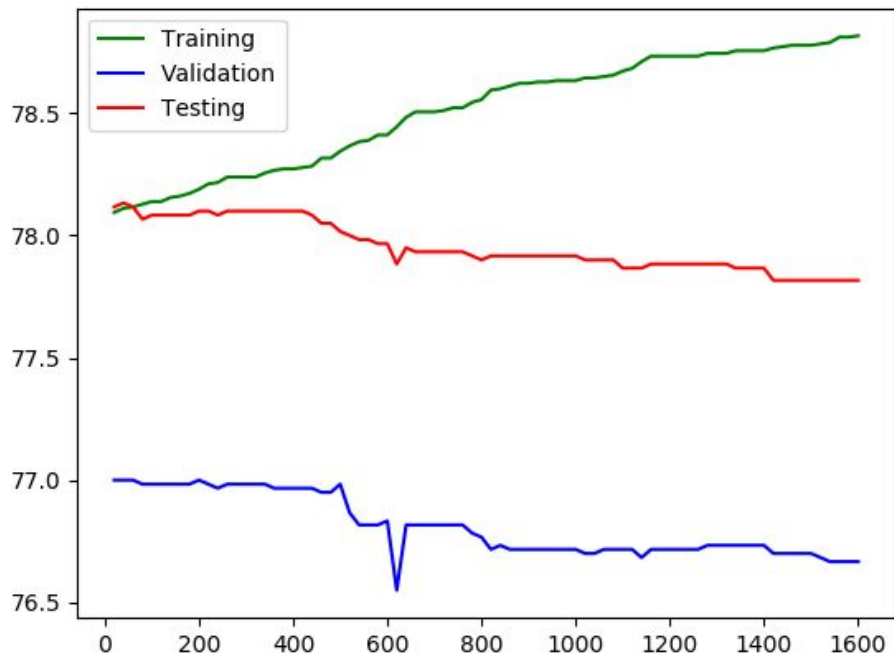


sklearn's
`train_test_split`
function mislead people
to **skip validation** as if it
is an optional or
unimportant split which
is not true!

And then people use
such “test” set as actual
validation set thus
skipping real test split
usage.



Example of test as validation



Even in popular sources people evaluate test set on each iteration which makes **test set become one more validation set**

<https://www.geeksforgeeks.org/machine-learning/difference-between-model-parameters-vs-hyperparameters/>



Size of splits

Subject to ad-hoc solutions in most of the cases. Split size controls tradeoff between training error and variance of performance estimation.

Dataset size	Recommended method	Typical ratio	Rationale
< 1,000 samples	K-fold CV (k=5–10)	N/A	Reduce variance due to small sample size
1k–100k	Train/val/test	60/20/20	Enough test data for stable estimate
100k–10M	Train/val/test	80/10/10 or 90/5/5	Data abundant; prioritize training
>10M	Holdout split	95/5 or 98/2	Test variance negligible

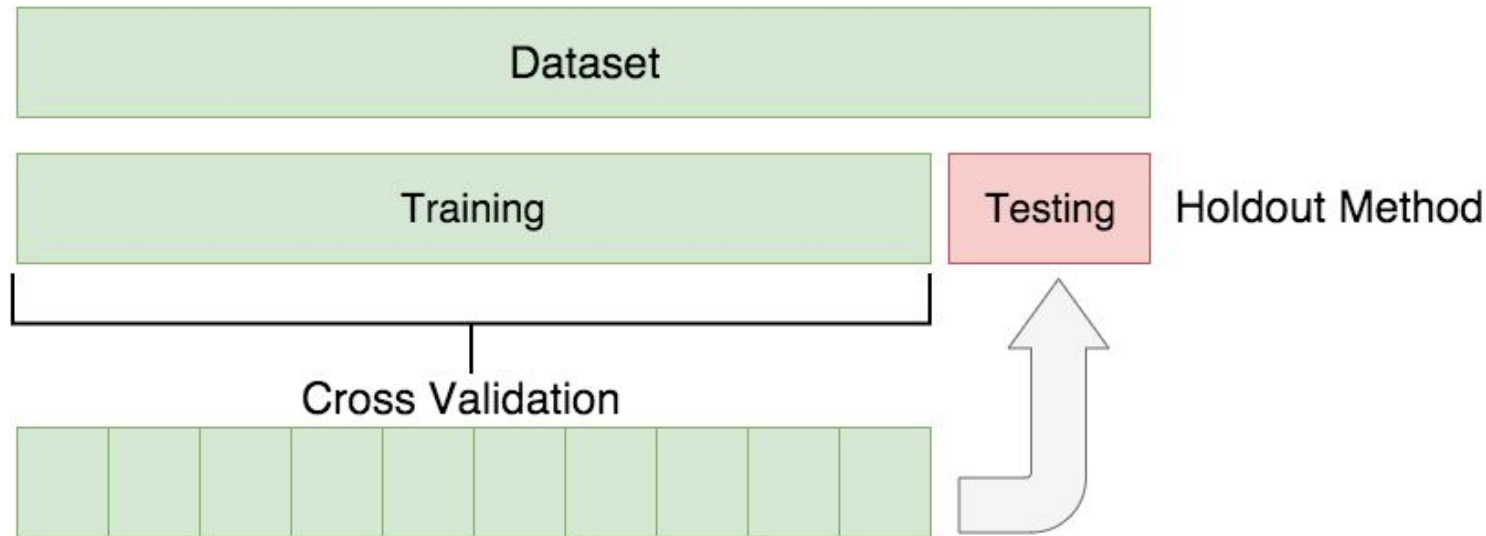


Stages organization

- fixed splits
 - works when fixed val and test samples are big enough to be representative
 - so today used in the vast majority of cases (~99%)
- cross-validation
 - efficient on small datasets because it makes validation set equal to train
 - used to be popular at age of small datasets
 - thought in most machine learning courses
 - edge case is Leave One Out (LOO) where number of splits equals to number of objects
- custom methods
 - whatever you compose for your case
 - can be pretty complex in some cases
 - e.g. uplift contain 5 validation sets

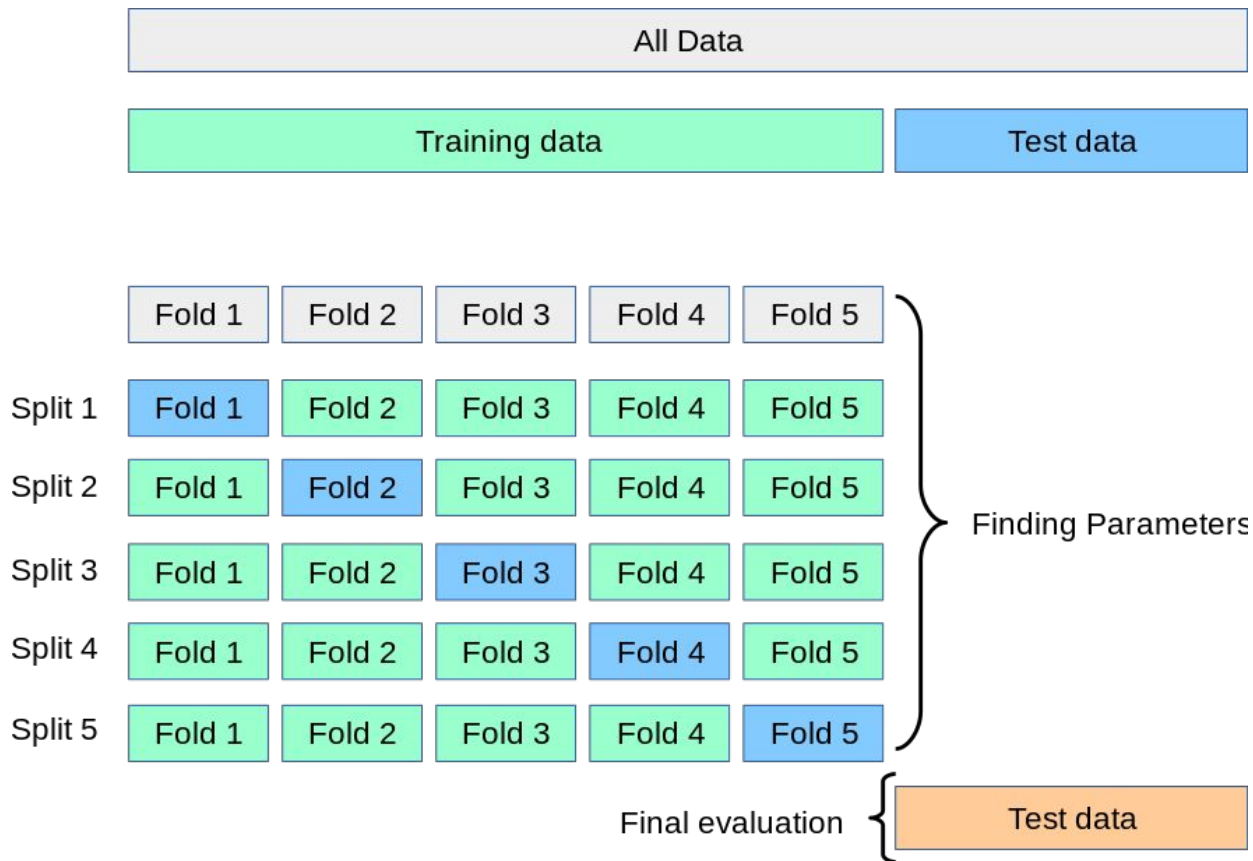


Cross-validation



In real life is used only on **small datasets** ($<10^4$ samples)

Cross-validation



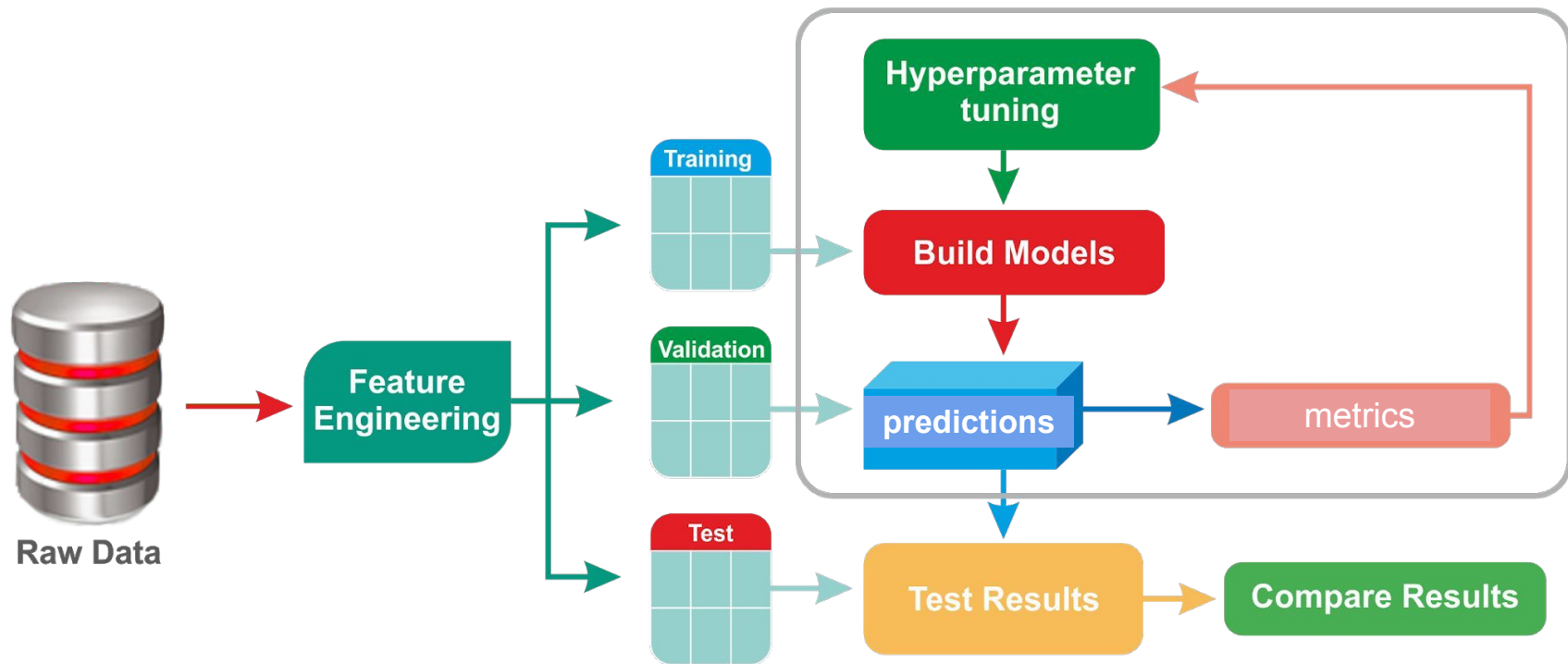


Validation summary

There are *two independent parts* of validation process:

- Split methods
 - uniform random
 - stratified
 - grouped
 - timeseries
 - custom
- Stages organization
 - fixed splits
 - cross-validation
 - custom

Stages of model training

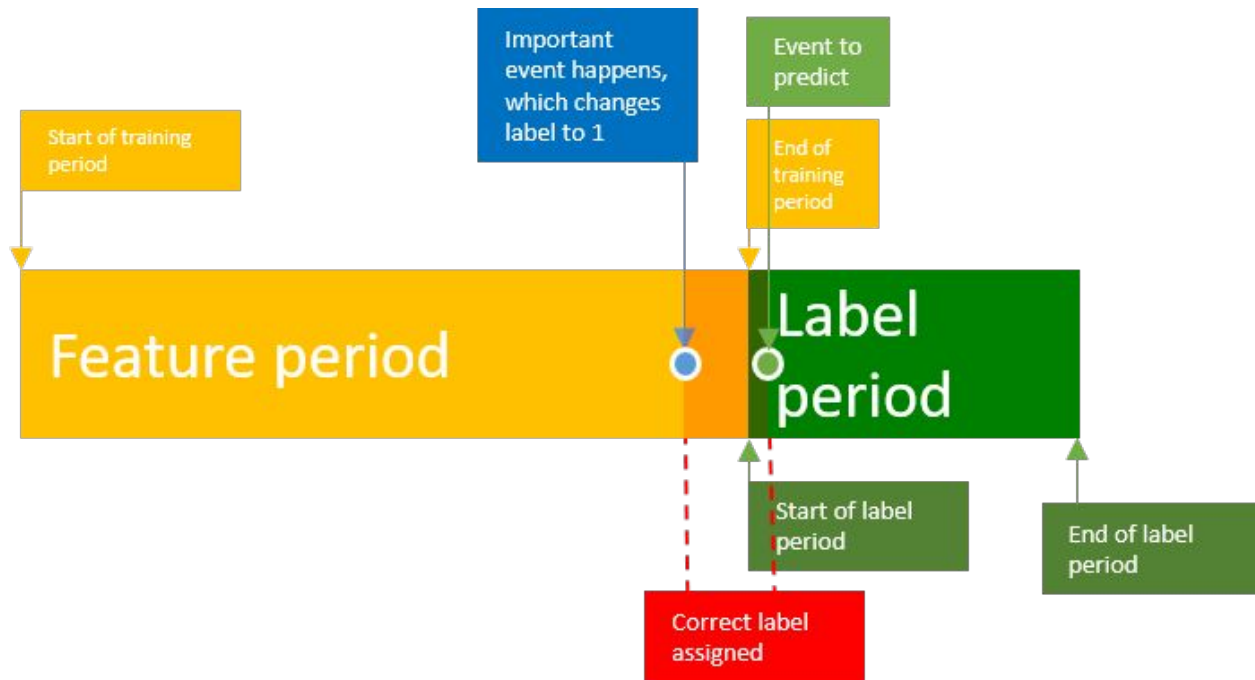


Although feature engineering better be done after split!

Data leaks



Improperly organized validation and evaluation may result in unfair information usage by model!



Outro



- Linear models are simple yet quite effective models
- Regularization incorporates some prior assumptions/additional constraints
- Trust your validation

Revise



1. Linear models overview
2. Linear Regression under the hood
3. Gauss-Markov theorem
4. Regularization in Linear regression
5. Model validation and evaluation

Thanks for attention!

Questions?

